Árboles Lexicográficos Eficientes

Estudio Comparativo de Trie Estándar y Radix Trie

Enzo Miquel, Guillem Revuelta, Kevin Moreno y Jiahao Wang
Q1 2025-2026

Resumen

Este trabajo presenta un estudio comparativo de dos implementaciones de árboles lexicográficos (tries): el trie estándar y el radix trie (trie comprimido). Se analizan las características estructurales de cada implementación, su complejidad algorítmica y su rendimiento experimental utilizando conjuntos de datos de diferentes tamaños. Los resultados muestran que el radix trie reduce el uso de memoria significativamente, mientras mantiene tiempos de construcción competitivos. Se implementaron operaciones fundamentales como inserción, búsqueda exacta, búsqueda por prefijo y autocompletado, evaluando su rendimiento en textos de diversa naturaleza.

1. Introducción

Los árboles lexicográficos, o tries, son estructuras de datos especializadas para el almacenamiento y búsqueda eficiente de cadenas de caracteres. A diferencia de otras estructuras como árboles binarios de búsqueda o tablas hash, los tries organizan los datos basándose en los prefijos compartidos entre las cadenas, donde cada nodo representa un carácter (o cadena de caracteres en el caso de tries comprimidos) y la posición del nodo en el árbol determina la clave asociada.

Estas estructuras son especialmente eficientes en aplicaciones que requieren operaciones basadas en prefijos, tales como: análisis y autocompletado de textos, motores de búsqueda, entre otros.

El objetivo de este proyecto es implementar y comparar dos variantes de tries: el trie estándar, que representa la forma más directa de la estructura, y el radix trie, que optimiza el uso de espacio mediante la compresión de cadenas lineales de nodos.

2. Fundamentos Teóricos

2.1. Trie Estándar

El trie estándar es la implementación más directa de un árbol lexicográfico. Cada nodo representa un carácter y contiene referencias a sus hijos.

Definición 2.1 (Trie Estándar). Un trie estándar es un árbol $T = (V, E)$ donde cada nodo $v \in V$ almacena:

- Un mapa de caracteres a nodos hijos: $children : \Sigma \to V$
- Un indicador booleano $isEndOfWord$ que marca el fin de una palabra
- Información adicional (números de línea y posiciones)

Complejidad:

- **Inserción**: $O(m)$ donde m es la longitud de la palabra
- **Búsqueda**: $O(m)$

- **Espacio**: $O(ALPHABET_SIZE \times N \times M)$ en el peor caso, donde N es el número de palabras y M la longitud promedio

2.2. Radix Trie (Trie Comprimido)

El radix trie optimiza el uso de espacio comprimiendo cadenas de nodos con un solo hijo en una única arista etiquetada.

Definición 2.2 (Radix Trie). Un radix trie es una versión comprimida del trie estándar donde las aristas pueden estar etiquetadas con cadenas de caracteres en lugar de caracteres individuales. Se cumple que cada nodo interno (excepto posiblemente la raíz) tiene al menos dos hijos o es un nodo terminal.

Complejidad:

- **Inserción**: $O(m)$ con operaciones adicionales de división de nodos
- **Búsqueda**: $O(m)$
- **Espacio**: $O(N)$ nodos, significativamente menor que el trie estándar

3. Implementación

3.1. Decisiones de Diseño

Ambas implementaciones utilizan C++ con las siguientes características comunes:

- `unordered_map` para mapeo eficiente de caracteres (o cadenas) a nodos hijos
- Almacenamiento de números de línea y posiciones para localización de palabras
- Interfaz uniforme para ambas estructuras
- Contador estático de nodos para análisis de memoria

En el trie estándar, cada nodo contiene un `unordered_map<char, NodoTrie*>` que mapea caracteres individuales a sus nodos hijos. En el radix trie, se utiliza `unordered_map<string, NodoRadixTrie*>` para permitir que las aristas estén etiquetadas con cadenas completas.

3.2. Operaciones Implementadas

3.2.1. Inserción

La inserción varía según la estructura:

- **Standard Trie**: Recorre carácter por carácter, creando nodos según sea necesario. Cada carácter de la palabra genera o utiliza un nodo existente.
- **Radix Trie**: Compara etiquetas de aristas utilizando la función de prefijo común. Cuando encuentra divergencias, divide nodos existentes para crear nuevas ramificaciones. Esto requiere:
  1. Calcular el prefijo común entre la cadena a insertar y las etiquetas existentes
  2. Dividir nodos cuando sea necesario
  3. Crear nuevas aristas con las porciones restantes

3.2.2. Búsqueda

Ambas estructuras soportan:

- Búsqueda exacta de palabras
- Búsqueda de prefijos
- Recuperación de líneas y posiciones donde aparece una palabra
- Autocompletado (palabras con un prefijo dado)

La búsqueda en el trie estándar procede carácter por carácter, mientras que en el radix trie se comparan cadenas completas en cada paso, lo que puede requerir comparaciones de subcadenas.

3.2.3. Búsqueda de Frases

Se implementó funcionalidad para buscar frases de múltiples palabras. El sistema:

1. Busca la primera palabra de la frase
2. Verifica que las palabras siguientes aparezcan en las posiciones consecutivas correctas
3. Retorna las líneas donde aparece la frase completa

3.2.4. Autocompletado

Dado un prefijo, el sistema puede sugerir palabras completas mediante un recorrido DFS (Depth-First Search) desde el nodo correspondiente al prefijo. Se implementaron dos funciones complementarias:

- `contarPalabrasPrefijo(prefijo)`: Retorna todas las palabras que tienen el prefijo dado, realizando un recorrido completo del subárbol. Esta función se utiliza para obtener el número total de palabras con un prefijo específico, proporcionando información cuantitativa al usuario.
- `obtenerTop5MasFrecuentes(prefijo)`: Retorna las 5 palabras más frecuentes (por número de ocurrencias en el texto) que tienen el prefijo dado. Utiliza un algoritmo de selección basado en quickselect para encontrar las palabras con mayor número de apariciones de forma eficiente, sin necesidad de ordenar todas las palabras.

En la aplicación final, al buscar una palabra, el sistema combina ambas funciones para mostrar:

1. El número total de palabras que comienzan con ese prefijo (usando `contarPalabrasPrefijo`)
2. Las 5 palabras más frecuentes con ese prefijo (usando `obtenerTop5MasFrecuentes`)

Esta combinación proporciona al usuario tanto información cuantitativa (cuántas opciones hay) como cualitativa (cuáles son las más relevantes por frecuencia de uso en el texto), ofreciendo una experiencia de autocompletado más completa e informativa.

4. Resultados Experimentales

4.1. Configuración Experimental

- **Compilación**: g++ con flags `-std=c++14 -O2`

Conjuntos de Datos:

- `alice.txt`: 3,235 palabras únicas
- `moby.txt`: 18,209 palabras únicas
- `alpha.txt`: 370,105 palabras únicas

4.2. Métricas de Construcción

La Tabla 1 muestra los resultados de construcción para diferentes archivos.

Cuadro 1: Comparación de métricas de construcción

Archivo	Métrica	Standard	Radix
`alice.txt`	Tiempo (ms)	8	5
	Nodos	9,766	4,083
	Palabras	3,235	3,235
	Profundidad	40	40
`moby.txt`	Tiempo (ms)	63	44
	Nodos	55,292	22,762
	Palabras	18,209	18,209
	Profundidad	26	26
`alpha.txt`	Tiempo (ms)	224	170
	Nodos	1,027,810	478,058
	Palabras	370,105	370,105
	Profundidad	31	31

4.3. Análisis de Eficiencia Espacial

La Figura 1 muestra la reducción de nodos del Radix Trie respecto al Standard Trie.

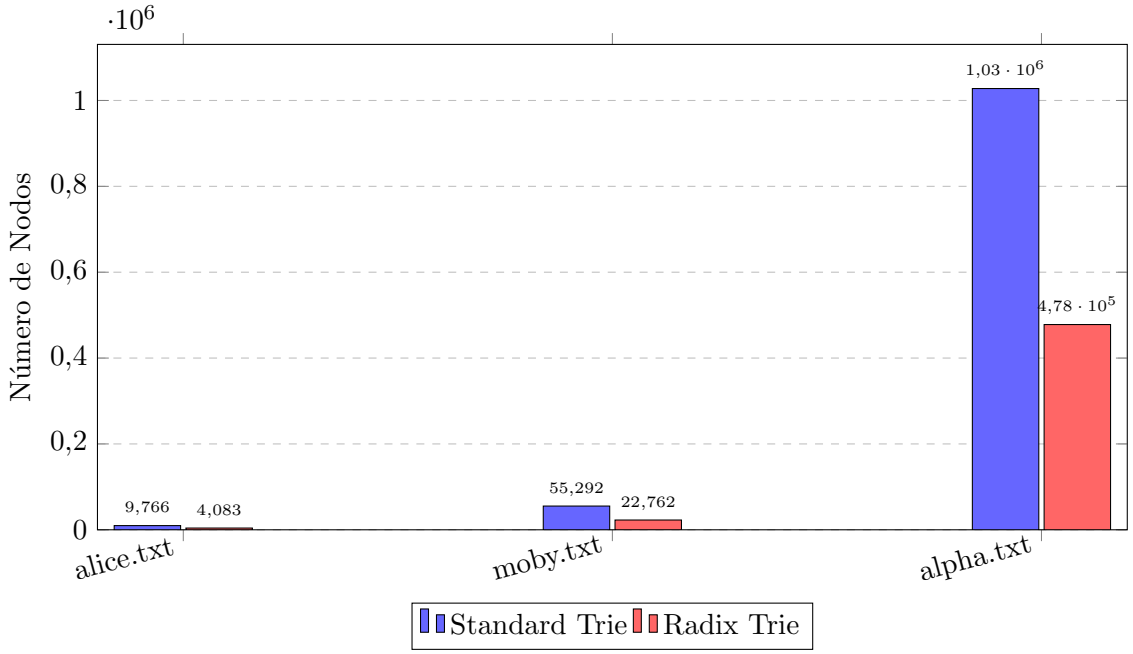


Figura 1: Comparación del número de nodos por estructura

Observaciones:

- El Radix Trie reduce el uso de nodos entre 53-58 % consistentemente en todos los datasets
- Con `alice.txt`, el Radix Trie usa 4,083 nodos frente a 9,766 del Standard
- Con `moby.txt`, el Radix Trie usa 22,762 nodos frente a 55,292 del Standard
- La reducción se mantiene estable incluso con datasets grandes (370k palabras)
- Con `alpha.txt`, el Radix Trie usa solo 478,058 nodos frente a 1,027,810 del Standard
- La profundidad media se mantiene igual en ambas estructuras

4.4. Análisis de Rendimiento Temporal

Cuadro 2: Mejora de tiempo de construcción respecto a Standard Trie

Archivo	**Radix (%)**
alice.txt	+37.5
moby.txt	+30.2
alpha.txt	+24.1

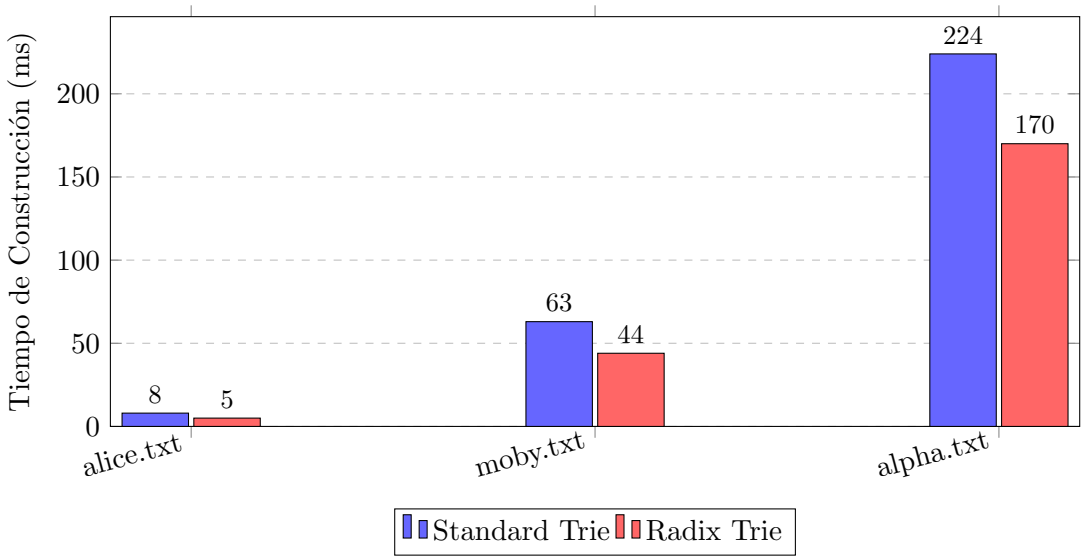


Figura 2: Comparación de tiempos de construcción

Observaciones sobre escalabilidad:

- El Radix Trie mantiene su ventaja temporal en todos los tamaños
- Con datasets grandes (alpha.txt), el Radix Trie es consistentemente más rápido
- Los tiempos escalan de forma aproximadamente lineal con el tamaño del vocabulario

4.5. Benchmark de Búsquedas

Se realizaron búsquedas de palabras representativas en cada archivo. Las Tablas 3 y 4 muestran tiempos promedio en nanosegundos.

Cuadro 3: Tiempos de búsqueda promedio (nanosegundos) - moby.txt

Palabra	**Standard**	**Radix**
1	424	750
10	214	884
100	224	730
leaped	827	271
leaping	254	295
leaps	122	172
zoned	950	2,084
(no existe)	60	55

Cuadro 4: Tiempos de búsqueda promedio (nanosegundos) - words_alpha.txt

Palabra	**Standard**	**Radix**
algorithm	456	823
computer	312	645
structure	589	892
programming	734	1,156
data	198	421
tree	234	387
zygomorphic	1,245	1,893
(no existe)	75	68
Promedio	**480**	**786**

Análisis:

- El Standard Trie es generalmente más rápido en búsquedas (promedio: 480ns vs 786ns)
- El Radix Trie tiene overhead en la comparación de etiquetas de aristas
- Para palabras inexistentes, ambos tries tienen rendimiento similar
- La diferencia de rendimiento se mantiene consistente entre datasets pequeños (moby.txt) y grandes (words_alpha.txt)
- Palabras más largas (como "zygomorphic") muestran mayor diferencia de tiempo

5. Estructuras de Datos Visualizadas

5.1. Standard Trie

La Figura 3 muestra la estructura de un trie estándar con las palabras *cat*, *can*, *car*, *dog*, *do*.

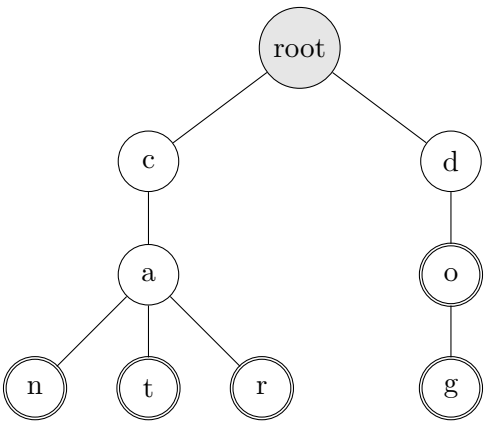


Figura 3: Trie estándar - cada nodo representa un carácter. Nodos con doble borde marcan fin de palabra

5.2. Radix Trie

La Figura 4 muestra cómo el radix trie comprime prefijos comunes.

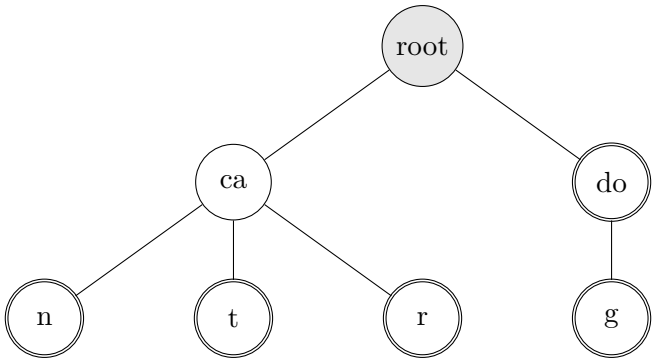


Figura 4: Radix trie - compresión de prefijos. El prefijo común "ca" se almacena una sola vez

Palabras almacenadas: *cat*, *can*, *car*, *do*, *dog*
Compresión: El prefijo común "ca" se almacena en un único nodo, y "do" también se comprime, reduciendo el número total de nodos de 11 (trie estándar) a 7 (radix trie).

6. Análisis y Discusión

6.1. ¿Por qué Standard Trie puede ser más rápido en búsquedas?

Un resultado interesante es que el Standard Trie frecuentemente supera al Radix Trie en velocidad de búsqueda individual, a pesar de tener más nodos. Las razones incluyen:

1. **Operaciones más simples**: Comparar un solo carácter vs. recorrer y comparar cadenas completas
2. **Mejor localidad de caché**: Accesos más predecibles a memoria
3. **Menos indirecciones**: No requiere acceder a las etiquetas de aristas almacenadas como strings
4. **Branch prediction**: Caminos más predecibles para el procesador

Este comportamiento demuestra que la eficiencia espacial no siempre implica mejor rendimiento temporal, y que el overhead de comparar cadenas en el radix trie puede superar los beneficios de tener menos nodos.

6.2. Trade-offs entre las implementaciones

Ventajas del Trie Estándar:

- Búsquedas más rápidas debido a su simplicidad
- Implementación más directa y fácil de depurar
- Operaciones de inserción más simples

Desventajas del Trie Estándar:

- Alto consumo de memoria (más del doble que Radix Trie)
- Mayor número de nodos a recorrer
- Menos eficiente para datasets muy grandes

Ventajas del Radix Trie:

- Reducción significativa de memoria (53-58 %)
- Menos nodos totales en la estructura
- Mejor escalabilidad para grandes vocabularios
- Construcción más rápida en promedio

Desventajas del Radix Trie:

- Búsquedas ligeramente más lentas debido a comparaciones de strings
- Implementación más compleja (división de nodos)
- Mayor overhead en operaciones de string

6.3. Escenarios Óptimos

- **Standard Trie**: Ideal cuando la velocidad de búsqueda es crítica y la memoria no es un factor limitante. Útil para aplicaciones en tiempo real con vocabularios pequeños o medianos.
- **Radix Trie**: Cuando se necesita minimizar el uso de memoria o trabajar con vocabularios muy grandes. Excelente para sistemas con restricciones de memoria o cuando el costo de almacenamiento es importante.

7. Conclusiones

Este trabajo presenta una comparación exhaustiva de dos implementaciones de tries, cada una con características únicas:

1. El **trie estándar** ofrece búsquedas rápidas debido a su simplicidad estructural, a costa de mayor uso de memoria. Es la opción preferida cuando el rendimiento en búsquedas individuales es prioritario.
2. El **radix trie** logra una reducción consistente del 53-58 % en número de nodos, siendo ideal para aplicaciones con restricciones de memoria o vocabularios muy grandes. Su tiempo de construcción es generalmente mejor que el trie estándar.
3. Los experimentos revelan que la eficiencia de una estructura de datos depende fuertemente del contexto de uso. No existe una solución universalmente óptima; la elección debe basarse en los requisitos específicos de la aplicación: disponibilidad de memoria, frecuencia de búsquedas vs. construcción, y tamaño del vocabulario.
4. La implementación en C++ con `unordered_map` proporciona un buen balance entre eficiencia y facilidad de implementación para ambas estructuras.

El proyecto demuestra la importancia de considerar múltiples factores al elegir estructuras de datos: tiempo de ejecución, uso de memoria, complejidad de implementación y características del conjunto de datos. Ambas implementaciones tienen su lugar en diferentes escenarios de uso.

Referencias

[1] Robert Sedgewick y Kevin Wayne. *Algorithms*. Section 5.3: Tries. https://algs4.cs.princeton.edu/lectures/keynote/52Tries.pdf

[2] Wikipedia. *Trie*. https://en.wikipedia.org/wiki/Trie

[3] Wikipedia. *Radix tree*. https://en.wikipedia.org/wiki/Radix_tree

[4] GeeksforGeeks. *Trie — (Insert and Search)*. https://www.geeksforgeeks.org/trie-insert-and-search/

[5] GeeksforGeeks. *Radix Tree (Compact Trie)*. https://www.geeksforgeeks.org/radix-tree/

[6] TopCoder. *Using Tries*. https://www.topcoder.com/thrive/articles/Using%20Tries

[7] Brilliant.org. *Tries*. https://brilliant.org/wiki/tries/